

Relatório do Projeto: Sistema de Chat Multi-Cliente

Disciplina: Redes de Computadores 2025.1

Universidade Federal de Alagoas (UFAL)

Grupo:

Donvicton Monteiro

Wictor Oliveira

2 de novembro de 2025

Conteúdo

1	Introdução	2
2	Principais Funcionalidades e Protocolos	2
2.1	Primitivas de Socket Utilizadas	2
2.2	Gerenciamento de Conexão e Nickname	2
2.3	Protocolo de Mensagens	3
3	Dificuldades Encontradas	3
4	O que poderia ter sido implementado a mais	4
5	Conclusão	4
6	Código Fonte	5
6.1	servidor.py	5
6.2	cliente.py	8

1 Introdução

Este relatório detalha o desenvolvimento de uma aplicação de chat multi-cliente, proposta pela disciplina de Redes de Computadores. O objetivo principal foi aplicar os conceitos fundamentais de programação com sockets, utilizando a linguagem Python, e implementar concorrência no servidor através do uso obrigatório da biblioteca `threading`.

A aplicação consiste em um servidor centralizado capaz de gerenciar múltiplas conexões de clientes simultaneamente. Os clientes podem se conectar, definir um apelido (nickname), enviar mensagens públicas para todos os participantes, enviar mensagens privadas para usuários específicos e listar os usuários online. O projeto foi desenvolvido estritamente com as primitivas de socket, sem o uso de Websockets.

2 Principais Funcionalidades e Protocolos

A comunicação entre cliente e servidor é baseada em TCP e segue um protocolo de aplicação simples, baseado em texto, que definimos para gerenciar o estado do chat.

2.1 Primitivas de Socket Utilizadas

O projeto utiliza estritamente as funções primitivas de socket, conforme exigido.

- `socket.socket()`: Usada tanto no servidor quanto no cliente para criar os endpoints de comunicação (sockets TCP/IP).
- `servidor_socket.bind()`: Vincula o servidor a um endereço IP e porta específicos (127.0.0.1:12345).
- `servidor_socket.listen()`: Coloca o servidor em modo de escuta, aguardando conexões.
- `servidor_socket.accept()`: Aguarda (de forma bloqueante) e aceita uma nova conexão de cliente, retornando um novo socket de conexão e o endereço do cliente.
- `cliente_socket.connect()`: Usada pelo cliente para estabelecer a conexão TCP com o servidor.
- `conexao.send()` / `conexao.sendall()`: Usadas para enviar dados (mensagens, comandos) pela rede.
- `conexao.recv()`: Usada para receber dados (de forma bloqueante), com um buffer de 1024 bytes.
- `conexao.close()`: Fecha a conexão do socket.

2.2 Gerenciamento de Conexão e Nickname

Para cada cliente que se conecta (retorno de `accept()`), o servidor instancia uma nova `Thread` para lidar exclusivamente com esse cliente. O protocolo de handshake de nickname é o seguinte:

1. Servidor envia ao cliente: "Digite seu nickname: "

2. Cliente responde com o nickname desejado.
3. O servidor (em uma seção crítica protegida por `threading.Lock`) verifica se o nickname já está em uso no dicionário de clientes.
4. Se estiver em uso, o servidor envia "[ERRO] Nickname inválido..." e volta ao passo 2.
5. Se for válido, o servidor armazena o cliente, envia uma mensagem de boas-vindas e notifica os demais usuários (broadcast) sobre a entrada do novo participante.

2.3 Protocolo de Mensagens

Após a validação, o cliente pode enviar strings que o servidor interpreta:

- **Mensagem Pública:** Se a string não começar com /, o servidor a formata como `<nickname> mensagem` e a retransmite (broadcast) para todos os outros clientes.
- **Comando /list:** O servidor localiza a thread do cliente e responde (apenas para ele) com "[SYSTEM] Usuários online: lista_de_nomes".
- **Comando /msg:** Se a string for `/msg <alvo> <texto>`, o servidor procura o socket do `<alvo>` em seu dicionário. Se encontrar, envia a mensagem formatada como "[PRIVADO de nickname] texto" apenas para o alvo. Se não encontrar, envia uma mensagem de erro ao remetente.

3 Dificuldades Encontradas

O principal desafio no desenvolvimento foi garantir a *thread safety* (segurança no uso de threads). Como o servidor cria uma thread para cada cliente, todas essas threads acessam o mesmo dicionário global `clientes`.

Condição de Corrida: Se dois clientes tentassem se registrar com o mesmo nickname ao mesmo tempo, ou se um cliente tentasse enviar uma mensagem privada para outro que estivesse se desconectando, poderíamos ter erros (condições de corrida). **Solução:** Isso foi resolvido utilizando um `threading.Lock` (`clientes_lock`). Qualquer operação de leitura ou escrita no dicionário `clientes` (adicionar, remover, iterar para broadcast) só é executada após a thread adquirir o *lock*, garantindo acesso exclusivo e atômico ao recurso compartilhado.

Desconexões Abruptas: Outro desafio foi tratar desconexões inesperadas (ex: cliente fecha o terminal). A chamada `recv()` nesses casos retorna 0 bytes ou levanta uma exceção (`ConnectionResetError`). **Solução:** Implementamos blocos `try...except...finally` para capturar essas exceções. A cláusula `finally` garante que a função `remover_cliente()` seja chamada, fechando o socket, removendo o usuário do dicionário (dentro de um *lock*) e notificando os demais sobre a saída.

4 O que poderia ter sido implementado a mais

Devido ao escopo e aos requisitos do projeto, focamos nas funcionalidades essenciais de um chat. Se tivéssemos mais tempo, as seguintes melhorias poderiam ser implementadas:

- **Canais/Salas:** Permitir que os usuários criem ou entrem em salas de chat separadas (ex: #geral, #jogos).
- **Transferência de Arquivos:** Adicionar um protocolo para que usuários possam enviar arquivos pequenos entre si.
- **Criptografia:** Usar a biblioteca `ssl` de Python para "envelopar" os sockets e garantir que a comunicação seja criptografada (SSL/TLS).
- **Interface Gráfica (GUI):** Substituir o cliente de terminal por uma interface gráfica (usando Tkinter, PyQt, etc.).

5 Conclusão

O projeto foi fundamental para aplicar na prática os conceitos teóricos de redes de computadores, especialmente a arquitetura cliente-servidor, o funcionamento do protocolo TCP/IP e a programação de sockets. O requisito de usar threads nos forçou a lidar com os desafios complexos da programação concorrente, como condições de corrida e a necessidade de *locks* para proteger dados compartilhados.

6 Código Fonte

Conforme a regra 1 do projeto, o código-fonte segue abaixo.

6.1 servidor.py

```
1 import socket
2 import threading
3
4 HOST = '127.0.0.1' # Localhost
5 PORTA = 12345
6
7 # Dicionário para armazenar os clientes conectados
8 # A chave será o socket do cliente, o valor será o nickname
9 clientes = {}
10 # Lock para proteger o acesso ao dicionário 'clientes'
11 clientes_lock = threading.Lock()
12
13 def broadcast(mensagem, remetente_socket=None):
14     """ Envia uma mensagem para todos os clientes, exceto o
15     remetente. """
16     with clientes_lock:
17         for cliente_socket in clientes:
18             if cliente_socket != remetente_socket:
19                 try:
20                     cliente_socket.send(mensagem.encode('utf-8'))
21                 except socket.error:
22                     remover_cliente(cliente_socket)
23
24 def remover_cliente(cliente_socket):
25     """ Remove um cliente do dicionário de forma segura. """
26     if cliente_socket in clientes:
27         nickname = clientes.pop(cliente_socket)
28         cliente_socket.close()
29         print(f"[DESCONEX 0] {nickname} desconectou.")
30         broadcast(f"[SYSTEM] {nickname} saiu do chat.", None)
31
32 def lidar_com_cliente(conexao, endereco):
33     """ Função alvo da Thread: gerencia a conexão com um novo
34     cliente. """
35     print(f"[NOVA CONEX 0] {endereco} tentando se conectar...")
36
37     try:
38         # 1. Solicitar e validar o nickname
39         conexao.send("Digite seu nickname: ".encode('utf-8'))
40         nickname = conexao.recv(1024).decode('utf-8').strip()
41
42         with clientes_lock:
43             while not nickname or nickname in clientes.values():
44                 conexao.send("[ERRO] Nickname inválido ou já em
45                 uso. Tente outro: ".encode('utf-8'))
```

```

43         nickname = conexao.recv(1024).decode('utf-8').strip
44     ()
45
46         clientes[conexao] = nickname
47
48         # 2. Informar o cliente e avisar os outros
49         print(f"[CONEX O BEM-SUCEDIDA] {endereco} agora {
nickname}.".")
50         conexao.send(f"[SYSTEM] Bem-vindo, {nickname}!\n[SYSTEM]
Digite /list para ver usu rios online.\n[SYSTEM] Digite /msg <
nick> <mensagem> para chat privado.\n".encode('utf-8'))
51         broadcast(f"[SYSTEM] {nickname} entrou no chat.", conexao)
52
53         # 3. Loop principal para receber mensagens
54         while True:
55             dados = conexao.recv(1024)
56             if not dados:
57                 break
58
59             mensagem = dados.decode('utf-8').strip()
60
61             if mensagem.startswith('/list'):
62                 with clientes_lock:
63                     lista_usuarios = ", ".join(clientes.values())
64                     conexao.send(f"[SYSTEM] Usu rios online: {
lista_usuarios}\n".encode('utf-8'))
65
66             elif mensagem.startswith('/msg'):
67                 try:
68                     _, nick_alvo, msg_privada = mensagem.split(' ',
2)
69
70                     encontrado = False
71
72                     with clientes_lock:
73                         for socket_alvo, nick in clientes.items():
74                             if nick == nick_alvo:
75                                 socket_alvo.send(f"[PRIVADO de {
nickname}] {msg_privada}".encode('utf-8'))
76                                 conexao.send(f"[PRIVADO para {
nick_alvo}] {msg_privada}".encode('utf-8'))
77                                 encontrado = True
78                                 break
79
80                     if not encontrado:
81                         conexao.send(f"[SYSTEM] Erro: Usu rio '{
nick_alvo}' n o encontrado.".encode('utf-8'))
82
83                     except ValueError:
84                         conexao.send("[SYSTEM] Erro: Formato inv lido.
Use: /msg <nickname> <mensagem>".encode('utf-8'))

```

```

84         else:
85             print(f"[{nickname}] {mensagem}")
86             broadcast_msg = f"<{nickname}> {mensagem}"
87             broadcast(broadcast_msg, conexao)
88
89     except (ConnectionResetError, ConnectionAbortedError):
90         print(f"[ERRO] Conex o com {nickname} ({endereco}) perdida
91 abruptamente.")
92     finally:
93         with clientes_lock:
94             remover_cliente(conexao)
95
96 def iniciar_servidor():
97     """ Fun o principal para iniciar o servidor de chat. """
98
99     servidor_socket = socket.socket(socket.AF_INET, socket.
100 SOCK_STREAM)
101     servidor_socket.setsockopt(socket.SOL_SOCKET, socket.
102 SO_REUSEADDR, 1)
103
104     try:
105         servidor_socket.bind((HOST, PORTA))
106         servidor_socket.listen(10)
107         print(f"[ESCUTANDO] Servidor de chat rodando em {HOST}:{
108 PORTA}")
109
110         while True:
111             conexao_cliente, endereco_cliente = servidor_socket.
112 accept()
113
114             thread_cliente = threading.Thread(target=
115 lidar_com_cliente, args=(conexao_cliente, endereco_cliente))
116             thread_cliente.daemon = True
117             thread_cliente.start()
118
119     except KeyboardInterrupt:
120         print("\n[DESLIGANDO] Desligando o servidor...")
121     finally:
122         with clientes_lock:
123             for cliente_socket in list(clientes.keys()):
124                 cliente_socket.send("[SYSTEM] O servidor est
125 sendo desligado.".encode('utf-8'))
126                 cliente_socket.close()
127             servidor_socket.close()
128             print("[FINALIZADO] Servidor desligado.")
129
130 if __name__ == "__main__":
131     iniciar_servidor()

```

Listing 1: Código fonte do servidor (servidor.py)

6.2 cliente.py

```
1 import socket
2 import threading
3 import sys
4
5 def receber_mensagens(cliente_socket):
6     """ Função alvo da Thread: Fica escutando por novas mensagens
    do servidor. """
7     while True:
8         try:
9             mensagem = cliente_socket.recv(1024).decode('utf-8')
10
11             if not mensagem:
12                 print("\n[CONEXÃO PERDIDA] Desconectado do
    servidor.")
13                 break
14
15                 print(f"{mensagem}")
16
17             except (ConnectionResetError, ConnectionAbortedError):
18                 print("\n[DESCONECTADO] A conexão com o servidor foi
    perdida.")
19                 break
20             except Exception as e:
21                 print(f"[ERRO] Erro ao receber mensagem: {e}")
22                 break
23
24             cliente_socket.close()
25             sys.exit()
26
27 def enviar_mensagens(cliente_socket):
28     """ Função para a Thread Principal: Lê o input e envia ao
    servidor. """
29     try:
30         while True:
31             mensagem = input()
32
33             if mensagem.lower() == 'sair':
34                 break
35
36             cliente_socket.sendall(mensagem.encode('utf-8'))
37
38             except EOFError:
39                 print("\nSaindo...")
40             except KeyboardInterrupt:
41                 print("\nSaindo...")
42             finally:
43                 cliente_socket.close()
44
45 def iniciar_cliente():
```



```

46     """ Função principal para iniciar o cliente de chat. """
47     HOST = input("Digite o IP do Servidor (default 127.0.0.1): ")
48     or '127.0.0.1'
49     PORTA = 12345
50
51     cliente_socket = socket.socket(socket.AF_INET, socket.
52     SOCK_STREAM)
53
54     try:
55         cliente_socket.connect((HOST, PORTA))
56     except ConnectionRefusedError:
57         print(f"[ERRO] Não foi possível conectar ao servidor em {
58         HOST}:{PORTA}.")
59         print("Verifique se o servidor está online e o IP/Porta
60         está correto.")
61         return
62     except Exception as e:
63         print(f"[ERRO DE CONEXÃO] {e}")
64         return
65
66     thread_recebimento = threading.Thread(target=receber_mensagens,
67     args=(cliente_socket,))
68     thread_recebimento.daemon = True
69     thread_recebimento.start()
70
71     enviar_mensagens(cliente_socket)
72
73 if __name__ == "__main__":
74     iniciar_cliente()

```

Listing 2: Código fonte do cliente (cliente.py)